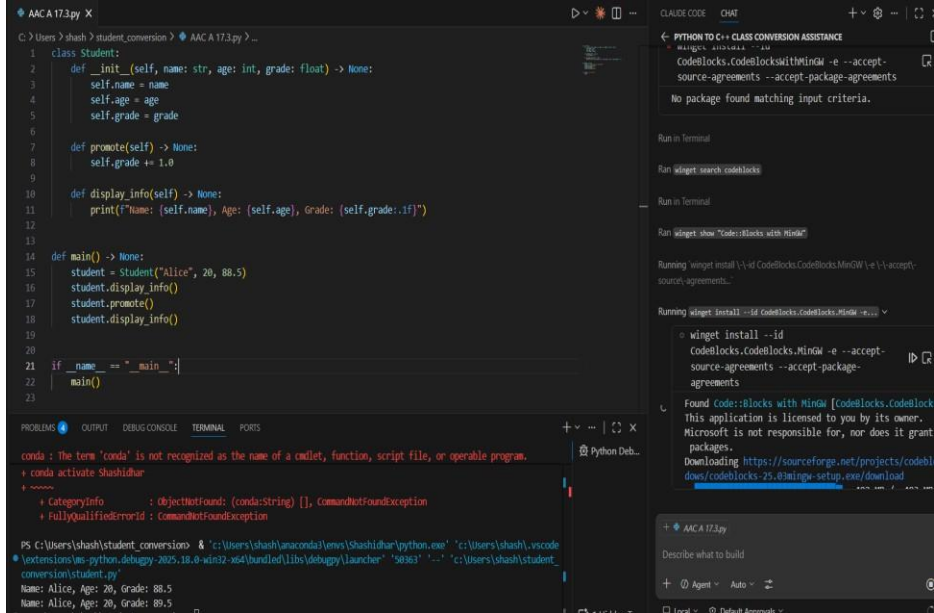


Name: shambhavi

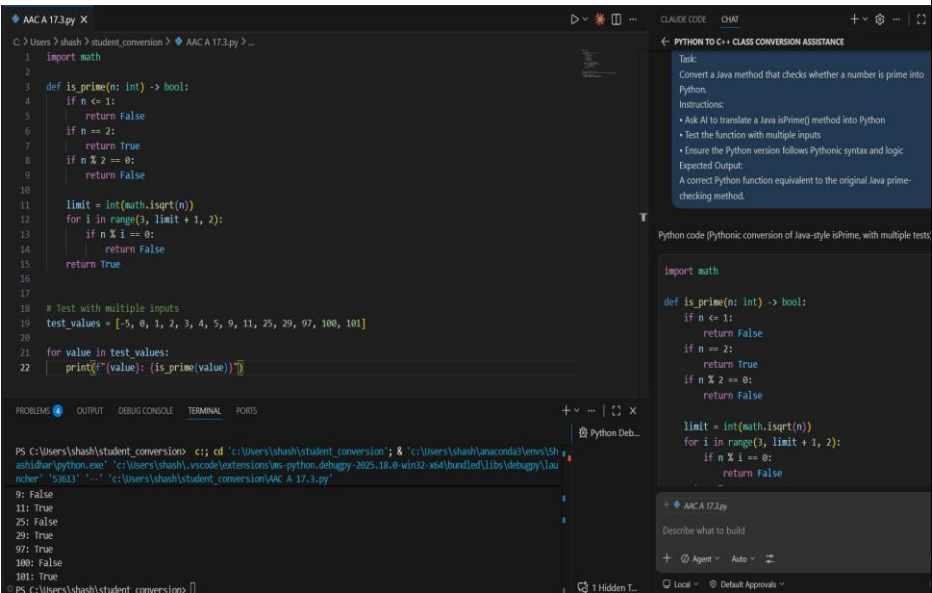
Hall ticket No:2303A51832

Batch:26

SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE		DEPARTMENT OF COMPUTER SCIENCE ENGINEERING	
Program Name: B. Tech		Assignment Type: Lab	Academic Year: 2025-2026
Course Coordinator Name		Dr. Rishabh Mittal	
Instructor(s) Name		Mr. S Naresh Kumar	
		Ms. B. Swathi	
		Dr. Sasanko Shekhar Gantayat	
		Mr. Md Sallauddin	
		Dr. Mathivanan	
		Mr. Y Srikanth	
		Ms. N Shilpa	
		Dr. Rishabh Mittal (Coordinator)	
		Dr. R. Prashant Kumar	
		Mr. Ankushavali MD	
		Mr. B Viswanath	
		Ms. Sujitha Reddy	
		Ms. A. Anitha	
		Ms. M.Madhuri	
		Ms. Katherashala Swetha	
Ms. Velpula sumalatha			
Mr. Bingi Raju			
CourseCode	23CS002PC304	Course Title	AI Assisted Coding
Year/Sem	III/II	Regulation	R23
Date and Day of Assignment	Week10 – Wednesday	Time(s)	23CSBTB01 To 23CSBTB52
Duration	2 Hours	Applicable to Batches	All batches
Assignment Number: 19.3(Present assignment number)/24(Total number of assignments)			
Q.No.	Question		Expected Time to complete
1	Lab 19 – Code Translation: Converting Between Programming Languages Lab Objectives: Understand how AI tools can assist in translating code between		Week10 – Wednesday

	different programming languages. - Learn to verify correctness and functionality after translation. - Explore syntactic and semantic differences between languages (e.g., Python, Java, C++). - Practice debugging and optimizing AI-translated code.	
	Task 1 – Python to C++ Conversion Task: Translate a simple Python class into C++ using AI assistance. Instructions: - Prompt AI to convert a Python class representing a Student with attributes and methods into equivalent C++ code - Ensure proper handling of: - Constructors - Data types - Access specifiers - Compile and run both versions to verify output consistency Expected Output: An equivalent working C++ class translated from Python. 	
	Task 2 – Java to Python Function Conversion Task: Convert a Java method that checks whether a number is prime into Python. Instructions: - Ask AI to translate a Java isPrime() method into Python - Test the function with multiple inputs - Ensure the Python version follows Pythonic syntax and logic	

Expected Output:
A correct Python function equivalent to the original Java prime-checking method.



Task 3 – Pseudocode to Python Implementation
Task:
Translate a given pseudocode algorithm into Python using AI.
Instructions:

- Provide AI with pseudocode for sorting numbers (e.g., bubble sort or selection sort)
- Ask AI to convert it into executable Python code
- Validate correctness using sample input lists

Expected Output:
A working Python program that correctly implements the pseudocode logic.

```

AAC A 17.3.py X
C:\Users\shash>shash>student_conversion> AAC A 17.3.py > ...
1 def bubble_sort(nums):
2     arr = nums[:]
3     n = len(arr)
4     for i in range(n):
5         swapped = False
6         for j in range(0, n - i - 1):
7             if arr[j] > arr[j + 1]:
8                 arr[j], arr[j + 1] = arr[j + 1], arr[j]
9                 swapped = True
10        if not swapped:
11            break
12    return arr
13
14
15 def main():
16     test_lists = [
17         [64, 34, 25, 12, 22, 11, 90],
18         [5, 1, 4, 2, 8],
19         [3, 3, 2, 1, 5, 0],
20         [],
21         [10],
22     ]
23
24     for lst in test_lists:
25         print("Input :", lst)
26         print("Sorted:", bubble_sort(lst))
27         print()
28
29 if __name__ == "__main__":
30     main()
31
32

```

Python Deb...

for lst in test_lists:
print("Input :", lst)
print("Sorted:", bubble_sort(lst))
print()

AAC A 17.3.py

Describe what to build

+ Agent - Auto -

PS C:\Users\shash\student_conversion> c:\cd 'c:\Users\shash\student_conversion'; & 'c:\Users\shash\anaconda3\envs\shashidhar\python.exe' 'c:\Users\shash\.vscode\extensions\ms-python.debugpy-2025.18.8-win12-x64\bundled\lib\debugpy\launcher' '56797' '--' 'c:\Users\shash\student_conversion\AAC A 17.3.py'

Input : []
Sorted: []

Input : [10]
Sorted: [10]

```

AAC A 17.3.py X
C:\Users\shash>shash>student_conversion> AAC A 17.3.py > ...
15 def main():
16     test_lists = [
17         [64, 34, 25, 12, 22, 11, 90],
18         [5, 1, 4, 2, 8],
19         [3, 3, 2, 1, 5, 0],
20         [],
21         [10],
22     ]
23
24     for lst in test_lists:
25         print("Input :", lst)
26         print("Sorted:", bubble_sort(lst))
27         print()
28
29 if __name__ == "__main__":
30     main()
31
32

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\shash\student_conversion> c;; cd 'c:\Users\shash\studashidhar\python.exe' 'c:\Users\shash\.vscode\extensions\ms-python.debugpy-2025.18.8-win12-x64\bundled\lib\debugpy\launcher' '56797' '--' 'c:\Users\shash\student_conversion\AAC A 17.3.py'

Input : []
Sorted: []

Input : [10]
Sorted: [10]

Task 4 – SQL to Pandas Query

Task:

Translate a SQL query into a Pandas DataFrame operation in Python.

Instructions:

- Provide AI with a SQL query (e.g., SELECT name, salary FROM employees WHERE salary > 50000).
- Ask AI to generate equivalent Pandas code.
- Test the generated code on a sample DataFrame.

Expected Output:

- Equivalent Pandas query that retrieves the correct subset of data.

The screenshot shows a VS Code editor with a Python file named `AAC A 17.3.py`. The code defines a pandas DataFrame with employee data and filters it for salaries greater than 50,000. The terminal window shows the command `python AAC A 17.3.py` being executed, resulting in the following output:

```
name salary
1 Bob 62000
2 Charlie 51000
4 Evan 75000
```

The sidebar on the right displays AI-generated code and instructions for Task 4, which involves translating a SQL query into a pandas DataFrame operation in Python. The instructions include:

- Provide AI with a SQL query (e.g., `SELECT name, salary FROM employees WHERE salary > 50000`).
- Ask AI to generate equivalent Pandas code.
- Test the generated code on a sample DataFrame.

The expected output is an equivalent Pandas query that retrieves the correct subset of data given code in python without comments.

Task 5 – Real-Time Application: Algorithm Translation Across Languages

Scenario:

A company maintains algorithms written in different programming languages.

Examples:

- Converting a Python searching algorithm into C
- Translating a Java sorting program into JavaScript

Instructions:

- Use AI to translate a selected algorithm between two programming languages
- Execute and test both versions
- Document translation challenges such as:
 - Syntax differences
 - Library support
 - Memory management

Expected Output:

Equivalent, tested code implementations in two different programming languages.

```
◆ AAC A17.3.py X
C:\Users\shash> student_conversion > AAC A17.3.py ...
1 def python_binary_search(arr, target):
2     left, right = 0, len(arr) - 1
3     while left <= right:
4         mid = left + (right - left) // 2
5         if arr[mid] == target:
6             return mid
7         if arr[mid] < target:
8             left = mid + 1
9         else:
10            right = mid - 1
11    return -1
12
13
14 def c_style_binary_search(arr, size, target):
15     left = 0
16     right = size - 1
17     while left <= right:
18         mid = left + (right - left) // 2
19         if arr[mid] == target:
20             return mid
21         elif arr[mid] < target:
22             left = mid + 1
23         else:
24             right = mid - 1
25    return -1
26
27
28 def java_bubble_sort(arr):
29     a = arr[:]
30     n = len(a)
31     for i in range(n - 1):
32         for j in range(0, n - i - 1):
33             if a[j] > a[j + 1]:
34                 temp = a[j]
35                 a[j] = a[j + 1]
36                 a[j + 1] = temp
37    return a
38
39
40 def java_quick_sort(arr):
41     a = arr[:]
42     n = len(a)
43     if n <= 1:
44         return a
45     pivot = a[n // 2]
46     left = []
47     right = []
48     for i in range(n):
49         if a[i] < pivot:
50             left.append(a[i])
51         elif a[i] > pivot:
52             right.append(a[i])
53         else:
54             continue
55     left = java_quick_sort(left)
56     right = java_quick_sort(right)
57     return left + [pivot] + right
58
59
60 def java_merge_sort(arr):
61     a = arr[:]
62     n = len(a)
63     if n <= 1:
64         return a
65     mid = n // 2
66     left = java_merge_sort(a[:mid])
67     right = java_merge_sort(a[mid:])
68     return merge(left, right)
69
70
71 def merge(left, right):
72     result = []
73     i = j = 0
74     while i < len(left) and j < len(right):
75         if left[i] <= right[j]:
76             result.append(left[i])
77             i += 1
78         else:
79             result.append(right[j])
80             j += 1
81     result.extend(left[i:])
82     result.extend(right[j:])
83     return result
84
85
86 def java_insert_sort(arr):
87     a = arr[:]
88     for i in range(1, len(a)):
89         j = i - 1
90         while j >= 0 and a[j] > a[j + 1]:
91             a[j], a[j + 1] = a[j + 1], a[j]
92             j -= 1
93     return a
94
95
96 def java_selection_sort(arr):
97     a = arr[:]
98     for i in range(len(a) - 1):
99         min_idx = i
100        for j in range(i + 1, len(a)):
101            if a[j] < a[min_idx]:
102                min_idx = j
103        a[i], a[min_idx] = a[min_idx], a[i]
104    return a
105
106
107 def java_heap_sort(arr):
108     a = arr[:]
109     n = len(a)
110     def heapify(i):
111         largest = i
112         left = 2 * i + 1
113         right = 2 * i + 2
114         if left < n and a[left] > a[largest]:
115             largest = left
116         if right < n and a[right] > a[largest]:
117             largest = right
118         if largest != i:
119             a[i], a[largest] = a[largest], a[i]
120             heapify(largest)
121     for i in range(n // 2 - 1, -1, -1):
122         heapify(i)
123     for i in range(n - 1, 0, -1):
124         a[i], a[0] = a[0], a[i]
125         heapify(0)
126     return a
127
128
129 def java_radix_sort(arr):
130     a = arr[:]
131     n = len(a)
132     if n <= 1:
133         return a
134     max_len = len(str(max(a)))
135     for i in range(max_len):
136         buckets = [[] for _ in range(10)]
137         for j in range(n):
138             digit = (a[j] // 10**i) % 10
139             buckets[digit].append(a[j])
140         a = []
141         for bucket in buckets:
142             a.extend(bucket)
143     return a
144
145
146 def java_counting_sort(arr):
147     a = arr[:]
148     n = len(a)
149     if n <= 1:
150         return a
151     min_val = min(a)
152     max_val = max(a)
153     count = [0] * (max_val - min_val + 1)
154     for i in range(n):
155         count[a[i] - min_val] += 1
156     output = []
157     for i in range(max_val - min_val + 1):
158         for j in range(count[i]):
159             output.append(min_val + i)
160     return output
161
162
163 def java_shell_sort(arr):
164     a = arr[:]
165     n = len(a)
166     gap = n // 2
167     while gap > 0:
168         for i in range(gap, n):
169             j = i - gap
170             while j >= 0 and a[j] > a[j + gap]:
171                 a[j], a[j + gap] = a[j + gap], a[j]
172                 j -= gap
173         gap = gap // 2
174     return a
175
176
177 def java_timsort(arr):
178     a = arr[:]
179     n = len(a)
180     if n <= 1:
181         return a
182     min_run = 32
183     runs = []
184     for i in range(n):
185         run = []
186         while i < n:
187             run.append(a[i])
188             i += min_run
189         runs.append(run)
190     for i in range(len(runs)):
191         runs[i] = java_insert_sort(runs[i])
192     return merge_sort(runs)
193
194
195 def java_double_sort(arr):
196     a = arr[:]
197     n = len(a)
198     for i in range(n // 2):
199         a[i], a[n - i - 1] = a[n - i - 1], a[i]
200     return java_merge_sort(a)
201
202
203 def java_quick_sort_recursive(arr, low, high):
204     if low < high:
205         pivot = arr[low + high]
206         left = []
207         right = []
208         for i in range(low, high):
209             if arr[i] < pivot:
210                 left.append(arr[i])
211             else:
212                 right.append(arr[i])
213         left = java_quick_sort_recursive(left, low, high)
214         right = java_quick_sort_recursive(right, low, high)
215         return left + [pivot] + right
216     return arr
217
218
219 def java_merge_sort_recursive(arr, low, high):
220     if low < high:
221         mid = (low + high) // 2
222         left = java_merge_sort_recursive(arr, low, mid)
223         right = java_merge_sort_recursive(arr, mid + 1, high)
224         return merge(left, right)
225     return arr
226
227
228 def java_selection_sort_recursive(arr, low, high):
229     if low < high:
230         min_idx = low
231         for i in range(low + 1, high):
232             if arr[i] < arr[min_idx]:
233                 min_idx = i
234         arr[low], arr[min_idx] = arr[min_idx], arr[low]
235         return java_selection_sort_recursive(arr, low + 1, high)
236     return arr
237
238
239 def java_heap_sort_recursive(arr, low, high):
240     if low < high:
241         largest = low
242         left = 2 * low + 1
243         right = 2 * low + 2
244         if left < high and arr[left] > arr[largest]:
245             largest = left
246         if right < high and arr[right] > arr[largest]:
247             largest = right
248         if largest != low:
249             arr[low], arr[largest] = arr[largest], arr[low]
250             return java_heap_sort_recursive(arr, largest, high)
251     return arr
252
253
254 def java_radix_sort_recursive(arr, exp):
255     n = len(arr)
256     buckets = [[] for _ in range(10)]
257     for i in range(n):
258         digit = (arr[i] // exp) % 10
259         buckets[digit].append(arr[i])
260     arr = []
261     for bucket in buckets:
262         arr.extend(bucket)
263     return java_radix_sort_recursive(arr, exp * 10)
264
265
266 def java_counting_sort_recursive(arr, min_val, max_val):
267     count = [0] * (max_val - min_val + 1)
268     for i in range(len(arr)):
269         count[arr[i] - min_val] += 1
270     output = []
271     for i in range(max_val - min_val + 1):
272         for j in range(count[i]):
273             output.append(min_val + i)
274     return output
275
276
277 def java_shell_sort_recursive(arr, gap):
278     for i in range(gap, len(arr)):
279         j = i - gap
280         while j >= 0 and arr[j] > arr[j + gap]:
281             arr[j], arr[j + gap] = arr[j + gap], arr[j]
282             j -= gap
283     gap = gap // 2
284     return java_shell_sort_recursive(arr, gap)
285
286
287 def java_timsort_recursive(arr, low, high):
288     if low < high:
289         mid = (low + high) // 2
290         left = java_timsort_recursive(arr, low, mid)
291         right = java_timsort_recursive(arr, mid + 1, high)
292         return merge(left, right)
293     return arr
294
295
296 def java_double_sort_recursive(arr, low, high):
297     if low < high:
298         mid = (low + high) // 2
299         left = java_double_sort_recursive(arr, low, mid)
300         right = java_double_sort_recursive(arr, mid + 1, high)
301         return merge(left, right)
302     return arr
303
304
305 def java_quick_sort_iterative(arr):
306     a = arr[:]
307     n = len(a)
308     stack = []
309     low = 0
310     high = n - 1
311     while low < high:
312         pivot = a[low + high]
313         left = []
314         right = []
315         for i in range(low, high):
316             if a[i] < pivot:
317                 left.append(a[i])
318             else:
319                 right.append(a[i])
320         left = java_quick_sort_iterative(left)
321         right = java_quick_sort_iterative(right)
322         a = left + [pivot] + right
323     return a
324
325
326 def java_merge_sort_iterative(arr):
327     a = arr[:]
328     n = len(a)
329     temp = [0] * n
330     for i in range(1, n // 2 + 1):
331         for j in range(0, n - i + 1, 2 * i):
332             left = a[j:j + i]
333             right = a[j + i:j + 2 * i]
334             merged = merge(left, right)
335             temp[j:j + 2 * i] = merged
336         a = temp[:n]
337     return a
338
339
340 def java_selection_sort_iterative(arr):
341     a = arr[:]
342     n = len(a)
343     for i in range(n - 1):
344         min_idx = i
345         for j in range(i + 1, n):
346             if a[j] < a[min_idx]:
347                 min_idx = j
348         a[i], a[min_idx] = a[min_idx], a[i]
349     return a
350
351
352 def java_heap_sort_iterative(arr):
353     a = arr[:]
354     n = len(a)
355     for i in range(n // 2 - 1, -1, -1):
356         largest = i
357         left = 2 * i + 1
358         right = 2 * i + 2
359         if left < n and a[left] > a[largest]:
360             largest = left
361         if right < n and a[right] > a[largest]:
362             largest = right
363         if largest != i:
364             a[i], a[largest] = a[largest], a[i]
365     for i in range(n - 1, 0, -1):
366         a[i], a[0] = a[0], a[i]
367     return a
368
369
370 def java_radix_sort_iterative(arr):
371     a = arr[:]
372     n = len(a)
373     max_len = len(str(max(a)))
374     for i in range(max_len):
375         buckets = [[] for _ in range(10)]
376         for j in range(n):
377             digit = (a[j] // 10**i) % 10
378             buckets[digit].append(a[j])
379         a = []
380         for bucket in buckets:
381             a.extend(bucket)
382     return a
383
384
385 def java_counting_sort_iterative(arr, min_val, max_val):
386     count = [0] * (max_val - min_val + 1)
387     for i in range(len(arr)):
388         count[arr[i] - min_val] += 1
389     output = []
390     for i in range(max_val - min_val + 1):
391         for j in range(count[i]):
392             output.append(min_val + i)
393     return output
394
395
396 def java_shell_sort_iterative(arr):
397     gap = len(arr) // 2
398     while gap > 0:
399         for i in range(gap, len(arr)):
400             j = i - gap
401             while j >= 0 and arr[j] > arr[j + gap]:
402                 arr[j], arr[j + gap] = arr[j + gap], arr[j]
403                 j -= gap
404         gap = gap // 2
405     return arr
406
407
408 def java_timsort_iterative(arr):
409     a = arr[:]
410     n = len(a)
411     min_run = 32
412     runs = []
413     for i in range(n):
414         run = []
415         while i < n:
416             run.append(a[i])
417             i += min_run
418         runs.append(run)
419     for i in range(len(runs)):
420         runs[i] = java_insert_sort_iterative(runs[i])
421     return merge_sort_iterative(runs)
422
423
424 def java_double_sort_iterative(arr):
425     a = arr[:]
426     n = len(a)
427     for i in range(n // 2):
428         a[i], a[n - i - 1] = a[n - i - 1], a[i]
429     return java_merge_sort_iterative(a)
430
431
432 def java_quick_sort_recursive(arr, low, high):
433     if low < high:
434         pivot = arr[low + high]
435         left = []
436         right = []
437         for i in range(low, high):
438             if arr[i] < pivot:
439                 left.append(arr[i])
440             else:
441                 right.append(arr[i])
442         left = java_quick_sort_recursive(left, low, high)
443         right = java_quick_sort_recursive(right, low, high)
444         return left + [pivot] + right
445     return arr
446
447
448 def java_merge_sort_recursive(arr, low, high):
449     if low < high:
450         mid = (low + high) // 2
451         left = java_merge_sort_recursive(arr, low, mid)
452         right = java_merge_sort_recursive(arr, mid + 1, high)
453         return merge(left, right)
454     return arr
455
456
457 def java_selection_sort_recursive(arr, low, high):
458     if low < high:
459         min_idx = low
460         for i in range(low + 1, high):
461             if arr[i] < arr[min_idx]:
462                 min_idx = i
463         arr[low], arr[min_idx] = arr[min_idx], arr[low]
464         return java_selection_sort_recursive(arr, low + 1, high)
465     return arr
466
467
468 def java_heap_sort_recursive(arr, low, high):
469     if low < high:
470         largest = low
471         left = 2 * low + 1
472         right = 2 * low + 2
473         if left < high and arr[left] > arr[largest]:
474             largest = left
475         if right < high and arr[right] > arr[largest]:
476             largest = right
477         if largest != low:
478             arr[low], arr[largest] = arr[largest], arr[low]
479             return java_heap_sort_recursive(arr, largest, high)
480     return arr
481
482
483 def java_radix_sort_recursive(arr, exp):
484     n = len(arr)
485     buckets = [[] for _ in range(10)]
486     for i in range(n):
487         digit = (arr[i] // exp) % 10
488         buckets[digit].append(arr[i])
489     arr = []
490     for bucket in buckets:
491         arr.extend(bucket)
492     return java_radix_sort_recursive(arr, exp * 10)
493
494
495 def java_counting_sort_recursive(arr, min_val, max_val):
496     count = [0] * (max_val - min_val + 1)
497     for i in range(len(arr)):
498         count[arr[i] - min_val] += 1
499     output = []
500     for i in range(max_val - min_val + 1):
501         for j in range(count[i]):
502             output.append(min_val + i)
503     return output
504
505
506 def java_shell_sort_recursive(arr, gap):
507     for i in range(gap, len(arr)):
508         j = i - gap
509         while j >= 0 and arr[j] > arr[j + gap]:
510             arr[j], arr[j + gap] = arr[j + gap], arr[j]
511             j -= gap
512     gap = gap // 2
513     return java_shell_sort_recursive(arr, gap)
514
515
516 def java_timsort_recursive(arr, low, high):
517     if low < high:
518         mid = (low + high) // 2
519         left = java_timsort_recursive(arr, low, mid)
520         right = java_timsort_recursive(arr, mid + 1, high)
521         return merge(left, right)
522     return arr
523
524
525 def java_double_sort_recursive(arr, low, high):
526     if low < high:
527         mid = (low + high) // 2
528         left = java_double_sort_recursive(arr, low, mid)
529         right = java_double_sort_recursive(arr, mid + 1, high)
530         return merge(left, right)
531     return arr
532
533
534 def java_quick_sort_iterative(arr):
535     a = arr[:]
536     n = len(a)
537     stack = []
538     low = 0
539     high = n - 1
540     while low < high:
541         pivot = a[low + high]
542         left = []
543         right = []
544         for i in range(low, high):
545             if a[i] < pivot:
546                 left.append(a[i])
547             else:
548                 right.append(a[i])
549         left = java_quick_sort_iterative(left)
550         right = java_quick_sort_iterative(right)
551         a = left + [pivot] + right
552     return a
553
554
555 def java_merge_sort_iterative(arr):
556     a = arr[:]
557     n = len(a)
558     temp = [0] * n
559     for i in range(1, n // 2 + 1):
560         for j in range(0, n - i + 1, 2 * i):
561             left = a[j:j + i]
562             right = a[j + i:j + 2 * i]
563             merged = merge(left, right)
564             temp[j:j + 2 * i] = merged
565         a = temp[:n]
566     return a
567
568
569 def java_selection_sort_iterative(arr):
570     a = arr[:]
571     n = len(a)
572     for i in range(n - 1):
573         min_idx = i
574         for j in range(i + 1, n):
575             if a[j] < a[min_idx]:
576                 min_idx = j
577         a[i], a[min_idx] = a[min_idx], a[i]
578     return a
579
580
581 def java_heap_sort_iterative(arr):
582     a = arr[:]
583     n = len(a)
584     for i in range(n // 2 - 1, -1, -1):
585         largest = i
586         left = 2 * i + 1
587         right = 2 * i + 2
588         if left < n and a[left] > a[largest]:
589             largest = left
590         if right < n and a[right] > a[largest]:
591             largest = right
592         if largest != i:
593             a[i], a[largest] = a[largest], a[i]
594     for i in range(n - 1, 0, -1):
595         a[i], a[0] = a[0], a[i]
596     return a
597
598
599 def java_radix_sort_iterative(arr):
600     a = arr[:]
601     n = len(a)
602     max_len = len(str(max(a)))
603     for i in range(max_len):
604         buckets = [[] for _ in range(10)]
605         for j in range(n):
606             digit = (a[j] // 10**i) % 10
607             buckets[digit].append(a[j])
608         a = []
609         for bucket in buckets:
610             a.extend(bucket)
611     return a
612
613
614 def java_counting_sort_iterative(arr, min_val, max_val):
615     count = [0] * (max_val - min_val + 1)
616     for i in range(len(arr)):
617         count[arr[i] - min_val] += 1
618     output = []
619     for i in range(max_val - min_val + 1):
620         for j in range(count[i]):
621             output.append(min_val + i)
622     return output
623
624
625 def java_shell_sort_iterative(arr):
626     gap = len(arr) // 2
627     while gap > 0:
628         for i in range(gap, len(arr)):
629             j = i - gap
630             while j >= 0 and arr[j] > arr[j + gap]:
631                 arr[j], arr[j + gap] = arr[j + gap], arr[j]
632                 j -= gap
633         gap = gap // 2
634     return arr
635
636
637 def java_timsort_iterative(arr):
638     a = arr[:]
639     n = len(a)
640     min_run = 32
641     runs = []
642     for i in range(n):
643         run = []
644         while i < n:
645             run.append(a[i])
646             i += min_run
647         runs.append(run)
648     for i in range(len(runs)):
649         runs[i] = java_insert_sort_iterative(runs[i])
650     return merge_sort_iterative(runs)
651
652
653 def java_double_sort_iterative(arr):
654     a = arr[:]
655     n = len(a)
656     for i in range(n // 2):
657         a[i], a[n - i - 1] = a[n - i - 1], a[i]
658     return java_merge_sort_iterative(a)
659
660
661 def java_quick_sort_recursive(arr, low, high):
662     if low < high:
663         pivot = arr[low + high]
664         left = []
665         right = []
666         for i in range(low, high):
667             if arr[i] < pivot:
668                 left.append(arr[i])
669             else:
670                 right.append(arr[i])
671         left = java_quick_sort_recursive(left, low, high)
672         right = java_quick_sort_recursive(right, low, high)
673         return left + [pivot] + right
674     return arr
675
676
677 def java_merge_sort_recursive(arr, low, high):
678     if low < high:
679         mid = (low + high) // 2
680         left = java_merge_sort_recursive(arr, low, mid)
681         right = java_merge_sort_recursive(arr, mid + 1, high)
682         return merge(left, right)
683     return arr
684
685
686 def java_selection_sort_recursive(arr, low, high):
687     if low < high:
688         min_idx = low
689         for i in range(low + 1, high):
690             if arr[i] < arr[min_idx]:
691                 min_idx = i
692         arr[low], arr[min_idx] = arr[min_idx], arr[low]
693         return java_selection_sort_recursive(arr, low + 1, high)
694     return arr
695
696
697 def java_heap_sort_recursive(arr, low, high):
698     if low < high:
699         largest = low
700         left = 2 * low + 1
701         right = 2 * low + 2
702         if left < high and arr[left] > arr[largest]:
703             largest = left
704         if right < high and arr[right] > arr[largest]:
705             largest = right
706         if largest != low:
707             arr[low], arr[largest] = arr[largest], arr[low]
708             return java_heap_sort_recursive(arr, largest, high)
709     return arr
710
711
712 def java_radix_sort_recursive(arr, exp):
713     n = len(arr)
714     buckets = [[] for _ in range(10)]
715     for i in range(n):
716         digit = (arr[i] // exp) % 10
717         buckets[digit].append(arr[i])
718     arr = []
719     for bucket in buckets:
720         arr.extend(bucket)
721     return java_radix_sort_recursive(arr, exp * 10)
722
723
724 def java_counting_sort_recursive(arr, min_val, max_val):
725     count = [0] * (max_val - min_val + 1)
726     for i in range(len(arr)):
727         count[arr[i] - min_val] += 1
728     output = []
729     for i in range(max_val - min_val + 1):
730         for j in range(count[i]):
731             output.append(min_val + i)
732     return output
733
734
735 def java_shell_sort_recursive(arr, gap):
736     for i in range(gap, len(arr)):
737         j = i - gap
738         while j >= 0 and arr[j] > arr[j + gap]:
739             arr[j], arr[j + gap] = arr[j + gap], arr[j]
740             j -= gap
741     gap = gap // 2
742     return java_shell_sort_recursive(arr, gap)
743
744
745 def java_timsort_recursive(arr, low, high):
746     if low < high:
747         mid = (low + high) // 2
748         left = java_timsort_recursive(arr, low, mid)
749         right = java_timsort_recursive(arr, mid + 1, high)
750         return merge(left, right)
751     return arr
752
753
754 def java_double_sort_recursive(arr, low, high):
755     if low < high:
756         mid = (low + high) // 2
757         left = java_double_sort_recursive(arr, low, mid)
758         right = java_double_sort_recursive(arr, mid + 1, high)
759         return merge(left, right)
760     return arr
761
762
763 def java_quick_sort_iterative(arr):
764     a = arr[:]
765     n = len(a)
766     stack = []
767     low = 0
768     high = n - 1
769     while low < high:
770         pivot = a[low + high]
771         left = []
772         right = []
773         for i in range(low, high):
774             if a[i] < pivot:
775                 left.append(a[i])
776             else:
777                 right.append(a[i])
778         left = java_quick_sort_iterative(left)
779         right = java_quick_sort_iterative(right)
780         a = left + [pivot] + right
781     return a
782
783
784 def java_merge_sort_iterative(arr):
785     a = arr[:]
786     n = len(a)
787     temp = [0] * n
788     for i in range(1, n // 2 + 1):
789         for j in range(0, n - i + 1, 2 * i):
790             left = a[j:j + i]
791             right = a[j + i:j + 2 * i]
792             merged = merge(left, right)
793             temp[j:j + 2 * i] = merged
794         a = temp[:n]
795     return a
796
797
798 def java_selection_sort_iterative(arr):
799     a = arr[:]
800     n = len(a)
801     for i in range(n - 1):
802         min_idx = i
803         for j in range(i + 1, n):
804             if a[j] < a[min_idx]:
805                 min_idx = j
806         a[i], a[min_idx] = a[min_idx], a[i]
807     return a
808
809
810 def java_heap_sort_iterative(arr):
811     a = arr[:]
812     n = len(a)
813     for i in range(n // 2 - 1, -1, -1):
814         largest = i
815         left = 2 * i + 1
816         right = 2 * i + 2
817         if left < n and a[left] > a[largest]:
818             largest = left
819         if right < n and a[right] > a[largest]:
820             largest = right
821         if largest != i:
822             a[i], a[largest] = a[largest], a[i]
823     for i in range(n - 1, 0, -1):
824         a[i], a[0] = a[0], a[i]
825     return a
826
827
828 def java_radix_sort_iterative(arr):
829     a = arr[:]
830     n = len(a)
831     max_len = len(str(max(a)))
832     for i in range(max_len):
833         buckets = [[] for _ in range(10)]
834         for j in range(n):
835             digit = (a[j] // 10**i) % 10
836             buckets[digit].append(a[j])
837         a = []
838         for bucket in buckets:
839             a.extend(bucket)
840     return a
841
842
843 def java_counting_sort_iterative(arr, min_val, max_val):
844     count = [0] * (max_val - min_val + 1)
845     for i in range(len(arr)):
846         count[arr[i] - min_val] += 1
847     output = []
848     for i in range(max_val - min_val + 1):
849         for j in range(count[i]):
850             output.append(min_val + i)
851     return output
852
853
854 def java_shell_sort_iterative(arr):
855     gap = len(arr) // 2
856     while gap > 0:
857         for i in range(gap, len(arr)):
858             j = i - gap
859             while j >= 0 and arr[j] > arr[j + gap]:
860                 arr[j], arr[j + gap] = arr[j + gap], arr[j]
861                 j -= gap
862         gap = gap // 2
863     return arr
864
865
866 def java_timsort_iterative(arr):
867     a = arr[:]
868     n = len(a)
869     min_run = 32
870     runs = []
871     for i in range(n):
872         run = []
873         while i < n:
874             run.append(a[i])
875             i += min_run
876         runs.append(run)
877     for i in range(len(runs)):
878         runs[i] = java_insert_sort_iterative(runs[i])
879     return merge_sort_iterative(runs)
880
881
882 def java_double_sort_iterative(arr):
883     a = arr[:]
884     n = len(a)
885     for i in range(n // 2):
886         a[i], a[n - i - 1] = a[n - i - 1], a[i]
887     return java_merge_sort_iterative(a)
888
889
890 def java_quick_sort_recursive(arr, low, high):
891     if low < high:
892         pivot = arr[low + high]
893         left = []
894         right = []
895         for i in range(low, high):
896             if arr[i] < pivot:
897                 left.append(arr[i])
898             else:
899                 right.append(arr[i])
900         left = java_quick_sort_recursive(left, low, high)
901         right = java_quick_sort_recursive(right, low, high)
902         return left + [pivot] + right
903     return arr
904
905
906 def java_merge_sort_recursive(arr, low, high):
907     if low < high:
908         mid = (low + high) // 2
909         left = java_merge_sort_recursive(arr, low, mid)
910         right = java_merge_sort_recursive(arr, mid + 1, high)
911         return merge(left, right)
912     return arr
913
914
915 def java_selection_sort_recursive(arr, low, high):
916     if low < high:
917         min_idx = low
918         for i in range(low + 1, high):
919             if arr[i] < arr[min_idx]:
920                 min_idx = i
921         arr[low], arr[min_idx] = arr[min_idx], arr[low]
922         return java_selection_sort_recursive(arr, low + 1, high)
923     return arr
924
925
926 def java_heap_sort_recursive(arr, low, high):
927     if low < high:
928         largest = low
929         left = 2 * low + 1
930         right = 2 * low + 2
931         if left < high and arr[left] > arr[largest]:
932             largest = left
933         if right < high and arr[right] > arr[largest]:
934             largest = right
935         if largest != low:
936             arr[low], arr[largest] = arr[largest], arr[low]
937             return java_heap_sort_recursive(arr, largest, high)
938     return arr
939
940
941 def java_radix_sort_recursive(arr, exp):
942     n = len(arr)
943     buckets = [[] for _ in range(10)]
944     for i in range(n):
945         digit = (arr[i] // exp) % 10
946         buckets[digit].append(arr[i])
947     arr = []
948     for bucket in buckets:
949         arr.extend(bucket)
950     return java_radix_sort_recursive(arr, exp * 10)
951
952
953 def java_counting_sort_recursive(arr, min_val, max_val):
954     count = [0] * (max_val - min_val + 1)
955     for i in range(len(arr)):
956         count[arr[i] - min_val] += 1
957     output = []
958     for i in range(max_val - min_val + 1):
959         for j in range(count[i]):
960             output.append(min_val + i)
961     return output
962
963
964 def java_shell_sort_recursive(arr, gap):
965     for i in range(gap, len(arr)):
966         j = i - gap
967         while j >= 0 and arr[j] > arr[j + gap]:
968             arr[j], arr[j + gap] = arr[j + gap], arr[j]
969             j -= gap
970     gap = gap // 2
971     return java_shell_sort_recursive(arr, gap)
972
973
974 def java_timsort_recursive(arr, low, high):
975     if low < high:
976         mid = (low + high) // 2
977         left = java_timsort_recursive(arr, low, mid)
978         right = java_timsort_recursive(arr, mid + 1, high)
979         return merge(left, right)
980     return arr
981
982
983 def java_double_sort_recursive(arr, low, high):
984     if low < high:
985         mid = (low + high) // 2
986         left = java_double_sort_recursive(arr, low, mid)
987         right = java_double_sort_recursive(arr, mid + 1, high)
988         return merge(left, right)
989     return arr
990
991
992 def java_quick_sort_iterative(arr):
993     a = arr[:]
994     n = len(a)
995     stack = []
996     low = 0
997     high = n - 1
998     while low < high:
999         pivot = a[low + high]
1000        left = []
1001        right = []
1002        for i in range(low, high):
1003            if a[i] < pivot:
1004                left.append(a[i])
1005            else:
1006                right.append(a[i])
1007        left = java_quick_sort_iterative(left)
1008        right = java_quick_sort_iterative(right)
1009        a = left + [pivot] + right
1010    return a
1011
1012
1013 def java_merge_sort_iterative(arr):
1014     a = arr[:]
1015     n = len(a)
1016     temp = [0] * n
1017     for i in range(1, n // 2 + 1):
1018         for j in range(0, n - i + 1, 2 * i):
1019             left = a[j:j + i]
1020             right = a[j + i:j + 2 * i]
1021             merged = merge(left, right)
1022             temp[j:j + 2 * i] = merged
1023         a = temp[:n]
1024     return a
1025
1026
1027 def java_selection_sort_iterative(arr):
1028     a = arr[:]
1029     n = len(a)
1030     for i in range(n - 1):
1031         min_idx = i
1032         for j in range(i + 1, n):
1033             if a[j] < a[min_idx]:
1034                 min_idx = j
1035         a[i], a[min_idx] = a[min_idx], a[i]
1036     return a
1037
1038
1039 def java_heap_sort_iterative(arr):
1040     a = arr[:]
1041     n = len(a)
1042     for i in range(n // 2 - 1, -1, -1):
1043         largest = i
1044         left = 2 * i + 1
1045         right = 2 * i + 2
1046         if left < n and a[left] > a[largest]:
1047             largest = left
1048         if right < n and a[right] > a[largest]:
1049             largest = right
1050         if largest != i:
1051             a[i], a[largest] = a[largest], a[i]
1052     for i in range(n - 1, 0, -1):
1053         a[i], a[0] = a[0], a[i]
1054     return a
1055
1056
1057 def java_radix_sort_iterative(arr):
1058     a = arr[:]
1059     n = len(a)
1060     max_len = len(str(max(a)))
1061     for i in range(max_len):
1062         buckets = [[] for _ in range(10)]
1063         for j in range(n):
1064             digit = (a[j] // 10**i) % 10
1065             buckets[digit].append(a[j])
1066         a = []
1067         for bucket in buckets:
1068             a.extend(bucket)
1069     return a
1070
1071
1072 def java_counting_sort_iterative(arr, min_val, max_val):
1073     count = [0] * (max_val - min_val + 1)
1074     for i in range(len(arr)):
1075         count[arr[i] - min_val] += 1
1076     output = []
1077     for i in range(max_val - min_val + 1):
1078         for j in range(count[i]):
1079             output.append(min_val + i)
1080     return output
1081
1082
1
```

```
AACA 17.3.py X
C: > Users > shash > student_conversion > AACA 17.3.py > ...

40 def javascript_bubble_sort(arr):
41     a = arr[:]
42     n = len(a)
43     for i in range(n):
44         swapped = False
45         for j in range(0, n - i - 1):
46             if a[j] > a[j + 1]:
47                 a[j], a[j + 1] = a[j + 1], a[j]
48                 swapped = True
49         if not swapped:
50             break
51     return a
52
53
54 def run_search_tests():
55     data = [3, 7, 12, 19, 24, 31, 45, 58]
56     targets = [3, 24, 58, 9]
57     print("Search Tests")
58     for t in targets:
59         p = python_binary_search(data, t)
60         c = c_style_binary_search(data, len(data), t)
61         print(f"target={t}, python={p}, c_style={c}, match={p == c}")
62
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\shash\student_conversion> c++; cd 'c:\Users\shash\student_conversion'; & 'c:\Users\shash\anaconda3\envs\Shashidhar\python.exe' 'c:\Users\shash\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher' '49361' '--' 'c:\Users\shash\student_conversion\AAC A 17.3.py'
Search Tests
input=[64, 34, 25, 12, 22, 11, 90], java_like=[11, 12, 22, 25, 34, 64, 90], js_like=[11, 12, 22, 25, 34, 64, 90], match=True
input=[5, 1, 4, 2, 8], java_like=[1, 2, 4, 5, 8], js_like=[1, 2, 4, 5, 8], match=True
input=[3, 3, 2, 1, 5, 0], java_like=[0, 1, 2, 3, 3, 5], js_like=[0, 1, 2, 3, 3, 5], match=True
input=[], java_like=[], js_like=[], match=True
input=[10], java_like=[10], js_like=[10], match=True
```

```
AACA 17.3.py X
C: > Users > shash > student_conversion > AACA 17.3.py > ...

63
64 def run_sort_tests():
65     test_cases = [
66         [64, 34, 25, 12, 22, 11, 90],
67         [5, 1, 4, 2, 8],
68         [3, 3, 2, 1, 5, 0],
69         [],
70         [10],
71     ]
72     print("\nSort Tests")
73     for case in test_cases:
74         j = java_bubble_sort(case)
75         js = javascript_bubble_sort(case)
76         print(f"input={case}, java_like={j}, js_like={js}, match={j == js}")
77
78
79 if __name__ == "__main__":
80     run_search_tests()
81     run_sort_tests()
82
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\shash\student_conversion> c++; cd 'c:\Users\shash\student_conversion'; & 'c:\Users\shash\anaconda3\envs\Shashidhar\python.exe' 'c:\Users\shash\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher' '49361' '--' 'c:\Users\shash\student_conversion\AAC A 17.3.py'
Search Tests
input=[64, 34, 25, 12, 22, 11, 90], java_like=[11, 12, 22, 25, 34, 64, 90], js_like=[11, 12, 22, 25, 34, 64, 90], match=True
input=[5, 1, 4, 2, 8], java_like=[1, 2, 4, 5, 8], js_like=[1, 2, 4, 5, 8], match=True
input=[3, 3, 2, 1, 5, 0], java_like=[0, 1, 2, 3, 3, 5], js_like=[0, 1, 2, 3, 3, 5], match=True
input=[], java_like=[], js_like=[], match=True
input=[10], java_like=[10], js_like=[10], match=True
```

Note: Report should be submitted as a word document for all tasks in a single document with prompts, comments & code explanation, and output and if required, screenshots.